

ỨNG DỤNG PHÂN TÁN V-VI. Đồng bộ hoá

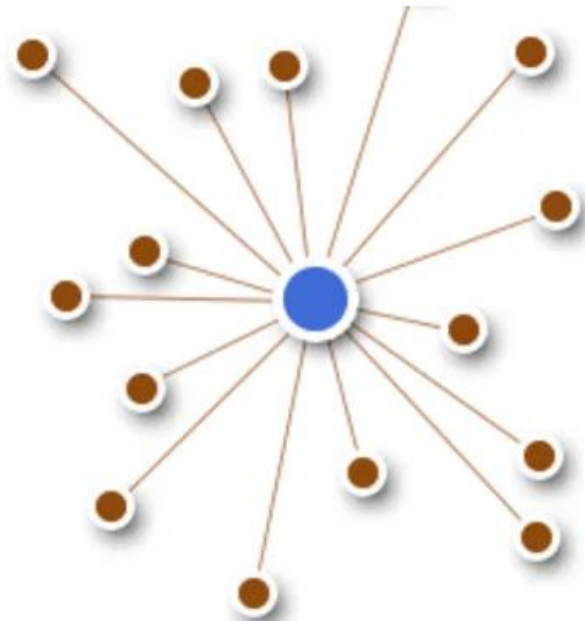
Nhóm chuyên môn UDPT – Trường Công nghệ thông tin Phenikaa
(Trần Đăng Hoan, Đỗ Quốc Trường, Nguyễn Thành Trung, Phạm Kim Thành, Nguyễn Hữu Đạt)

Nội dung

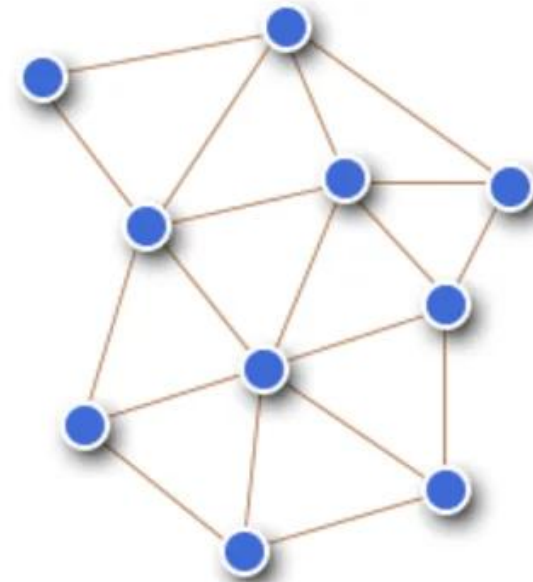
- Đồng bộ hóa đồng hồ vật lý
- Đồng bộ hóa đồng hồ logic
- Các thuật toán loại trừ lẫn nhau
- Các thuật toán bầu chọn

Mở đầu

- Trong một hệ tập trung, thời gian hệ thống là riêng biệt



Centralized

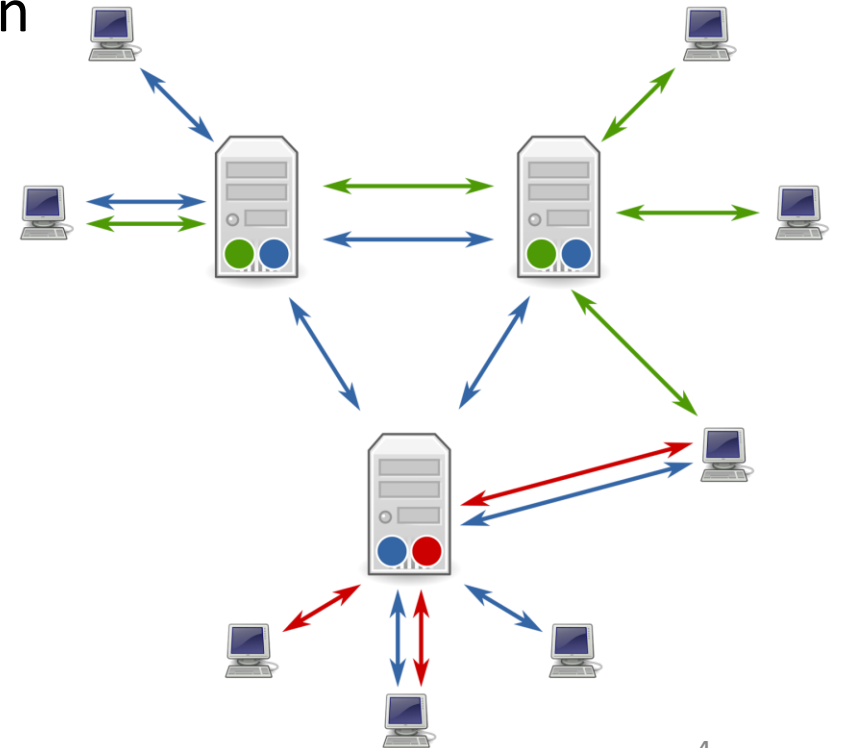


Distributed

- Trong hệ thống phân tán: đạt được sự thống nhất về thời gian ?

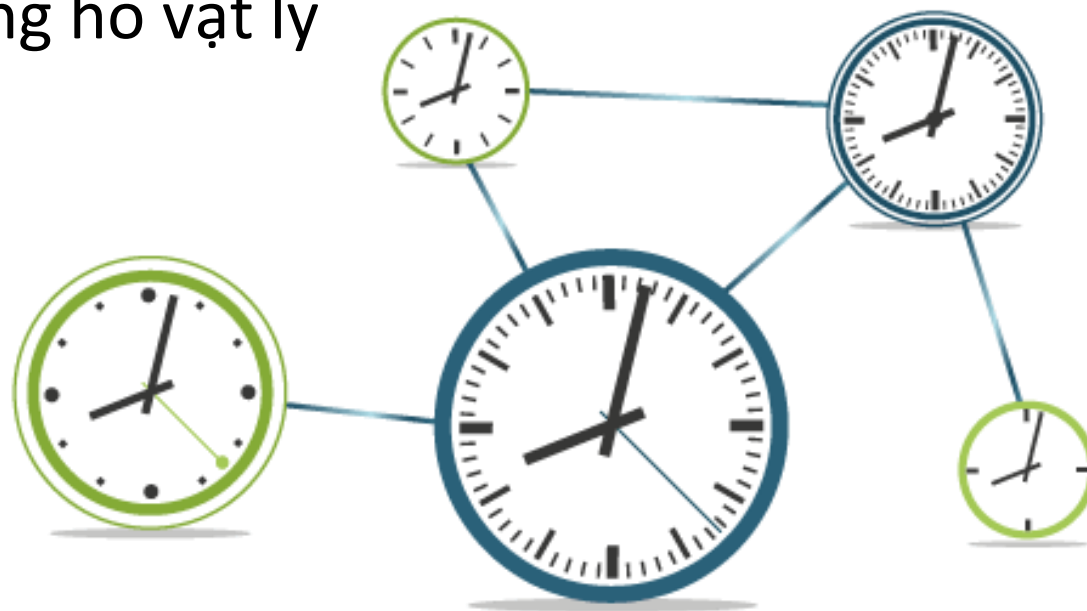
Mở đầu

- Các tiến trình thực hiện đồng bộ hóa ntn?
 - ▣ Nhiều tiến trình cần lần lượt vào sử dụng tài nguyên chia sẻ dùng chung
 - ▣ Nhiều tiến trình cần thống nhất thứ tự các sự kiện
 - ▣ Vấn đề đối với ngữ cảnh HPT?
- Đồng bộ hóa dựa trên giá trị thời gian thực
- Đồng bộ hóa dựa trên thứ tự các sự kiện

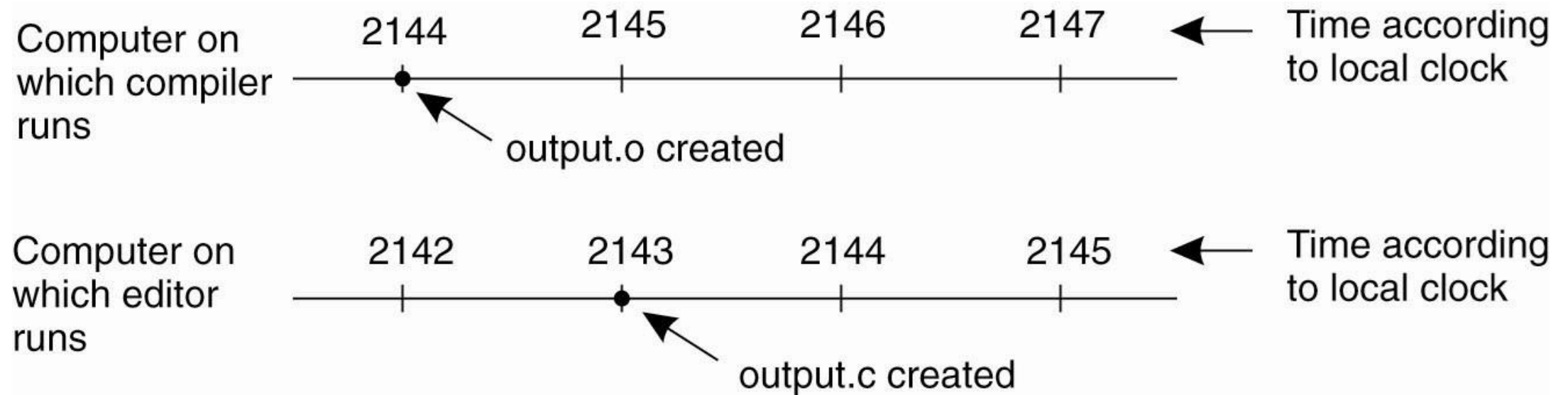


1. Đồng bộ hóa đồng hồ vật lý

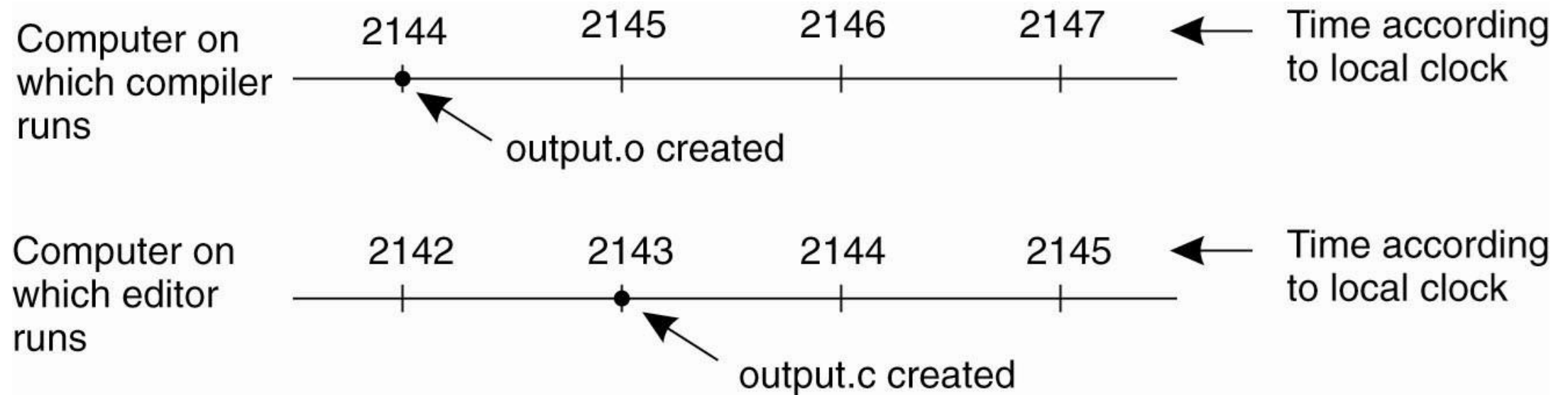
- Đồng hồ vật lý
- Các vấn đề khi không đồng bộ hóa đồng hồ vật lý
- Các thuật toán đồng bộ hóa đồng hồ vật lý



Ví dụ 1: Lập trình trong HPT



Ví dụ 1: Lập trình trong HPT



Đồng hồ vật lý (ĐHVL)

- Timer
- Counter & Holding register
- Clock tick
- Vấn đề trong HPT
 - ▣ Đồng bộ với giá trị thời gian vật lý thật
 - ▣ Đồng bộ các đồng hồ vật lý với nhau

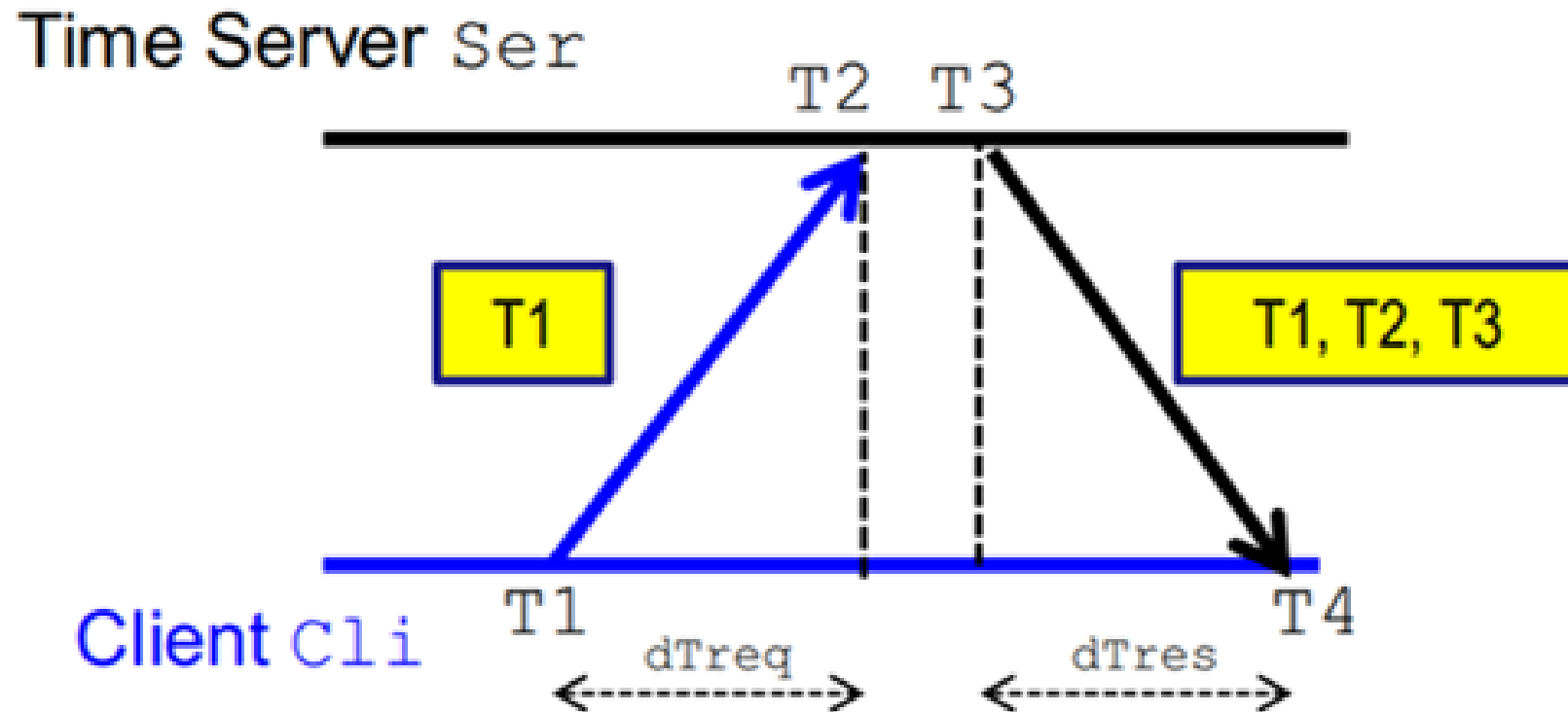


RTC IC
(Real Time Clock)

Các thuật toán đồng bộ hóa đồng hồ vật lý

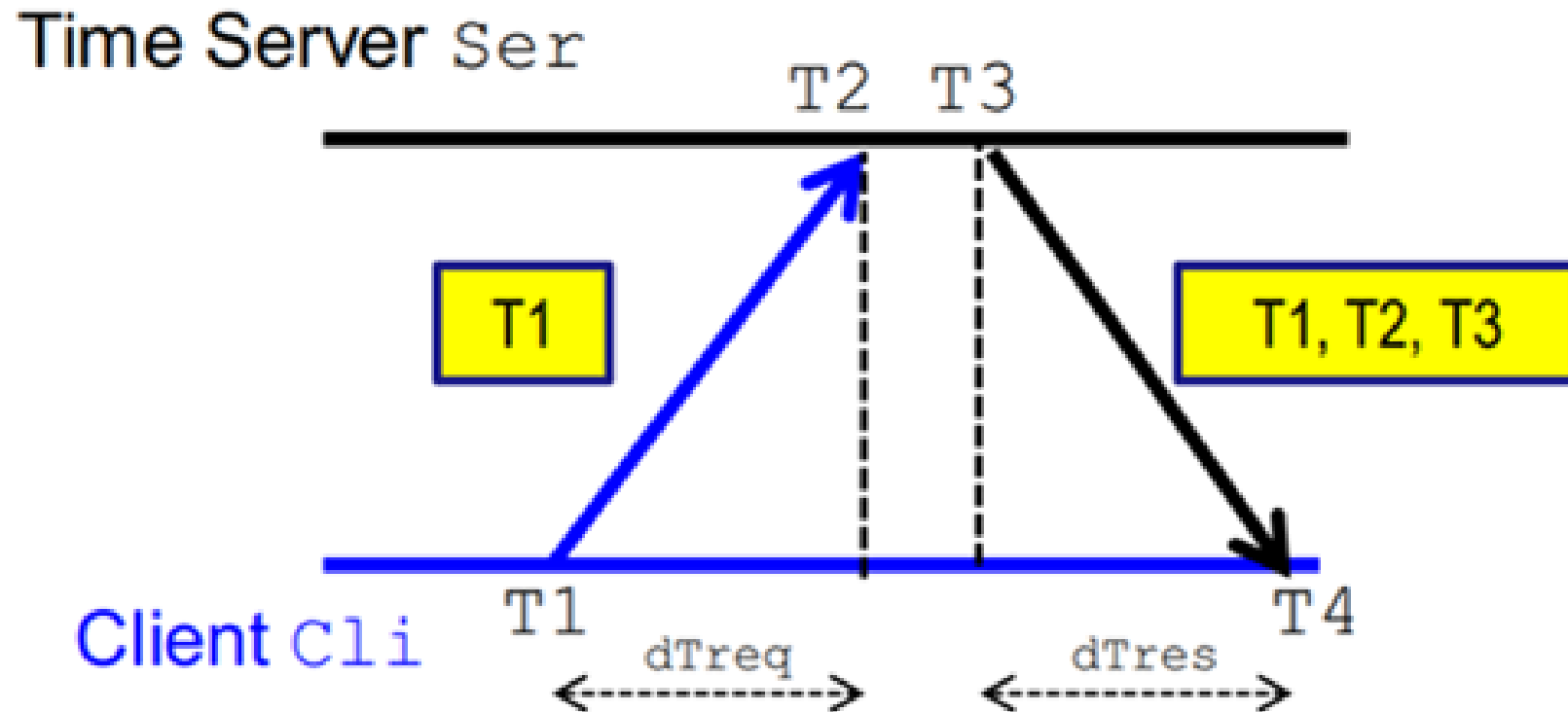
- Thuật toán Cristian
- Thuật toán Berkeley
- Đồng bộ hóa đồng hồ vật lý trong các mạng không dây

Cristian's Algorithm



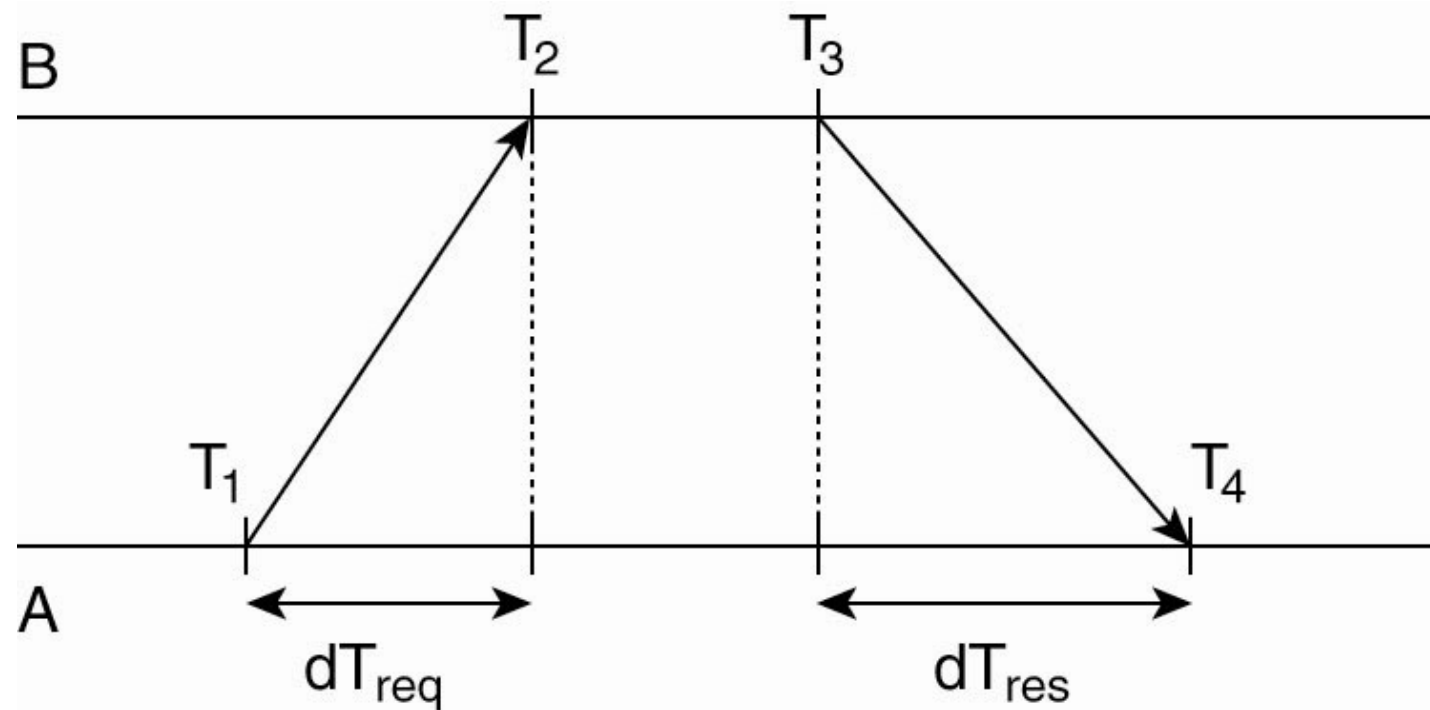
Getting the current time from a time server

Cristian's Algorithm



Getting the current time from a time server

Cristian's Algorithm



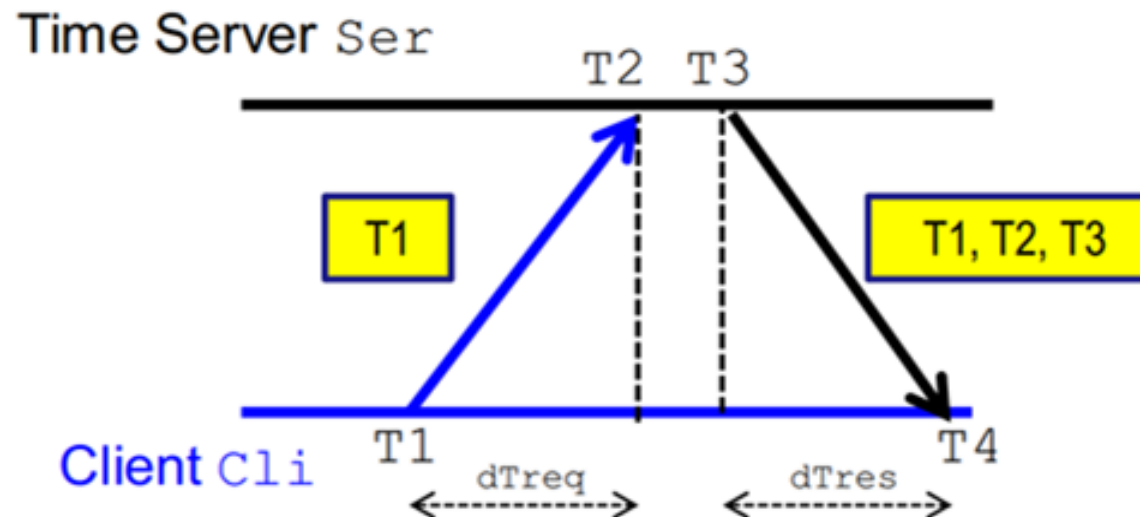
$$\theta = T_3 + \frac{(T_2 - T_1) + (T_4 - T_3)}{2} - T_4 = \frac{(T_2 - T_1) + (T_3 - T_4)}{2}$$

Cristian's Algorithm

Lúc 10:27:540 (giờ, phút, 1/100 giây), máy chủ B yêu cầu thời gian từ máy chủ thời gian A.

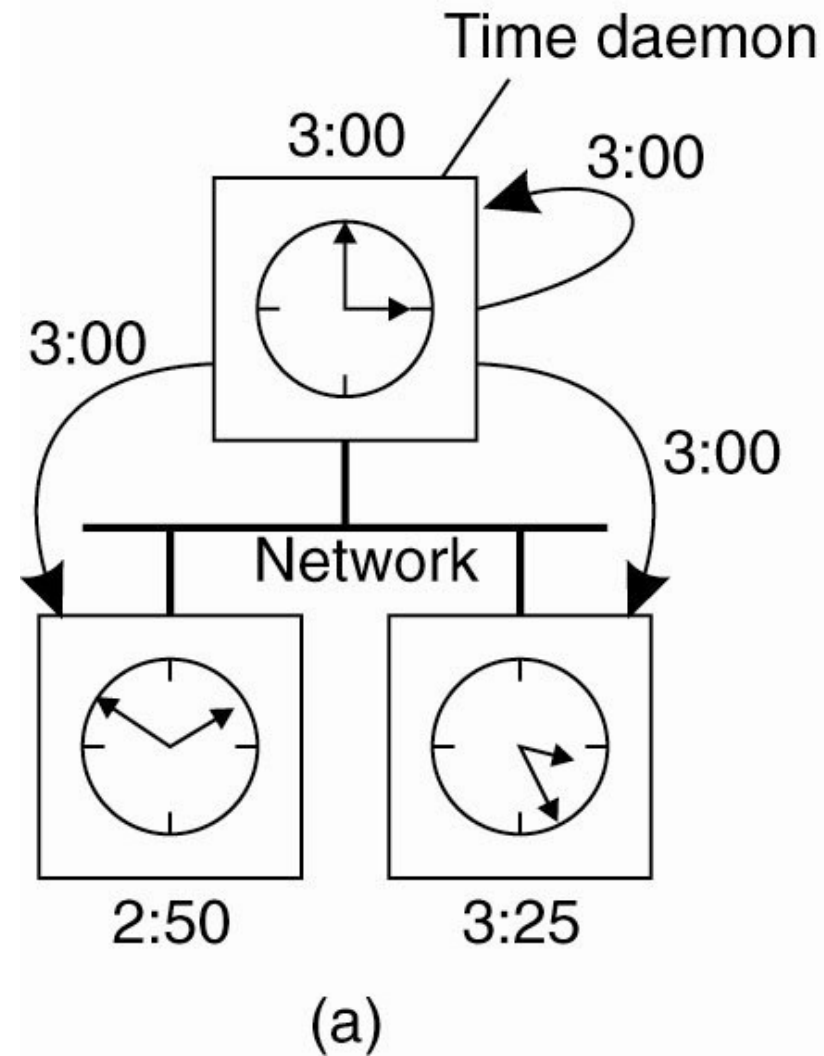
Lúc 10:27:610, máy chủ B nhận được phản hồi từ máy chủ thời gian A với dấu thời gian là 10:27:375.

Tìm độ lệch của đồng hồ B so với đồng hồ của máy chủ thời gian A (giả sử không có thời gian xử lý tại máy chủ thời gian cho dịch vụ thời gian)



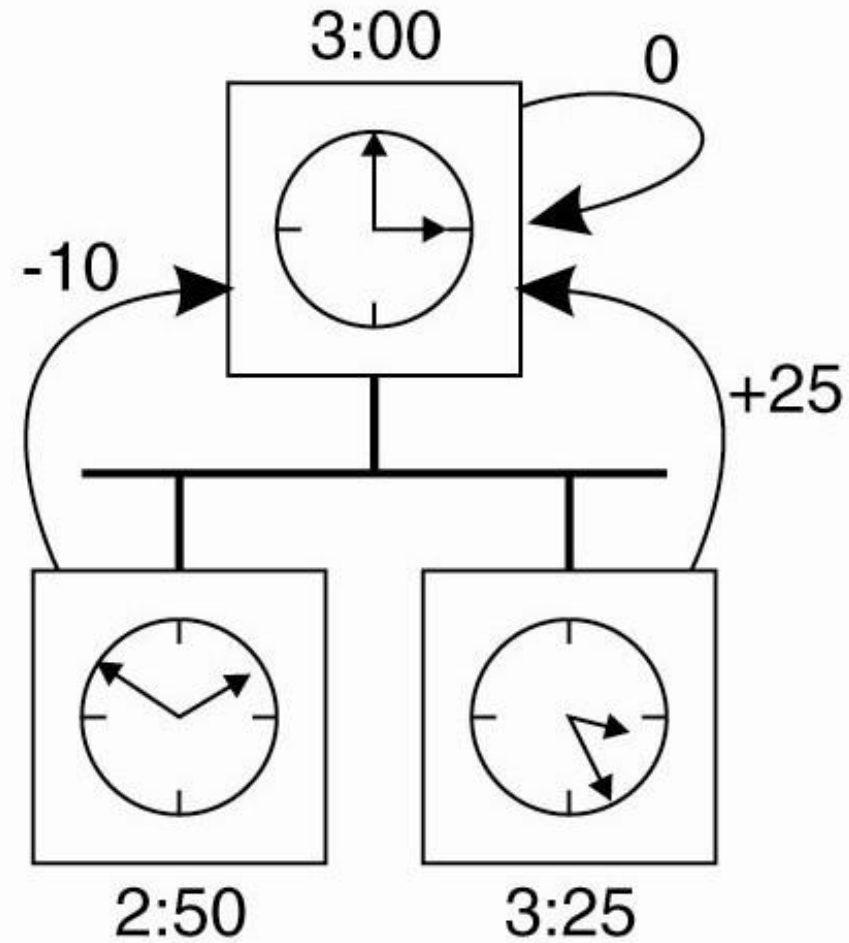
The Berkeley Algorithm (1)

- Time daemon quảng bá giá trị đồng hồ vật lý của mình



The Berkeley Algorithm (2)

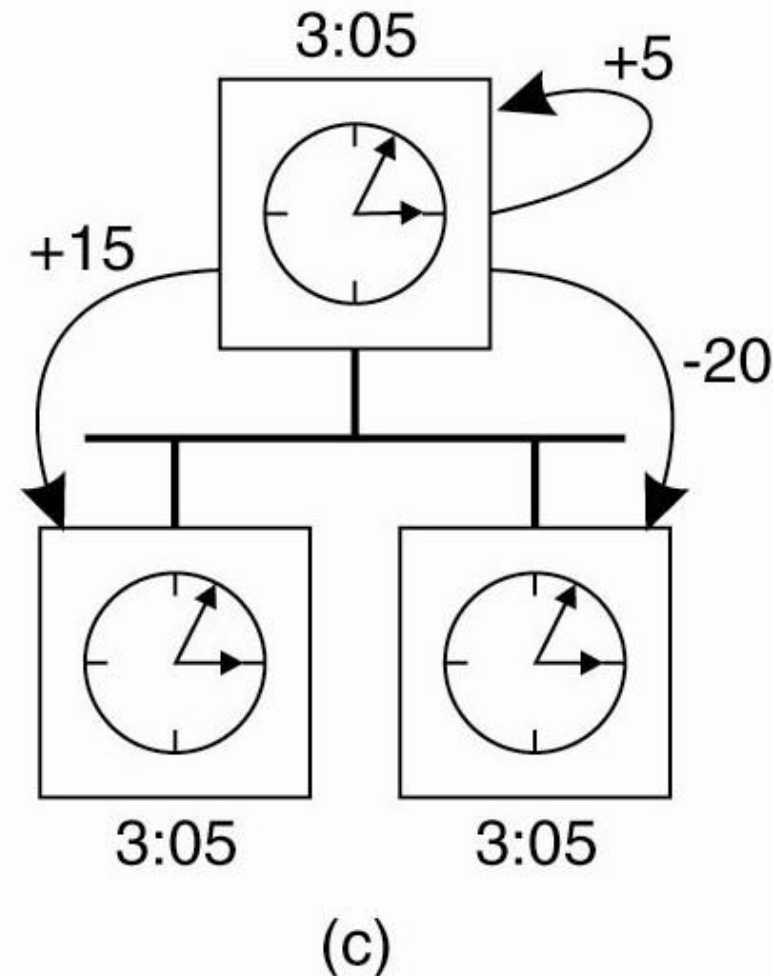
- các máy khác trả lời



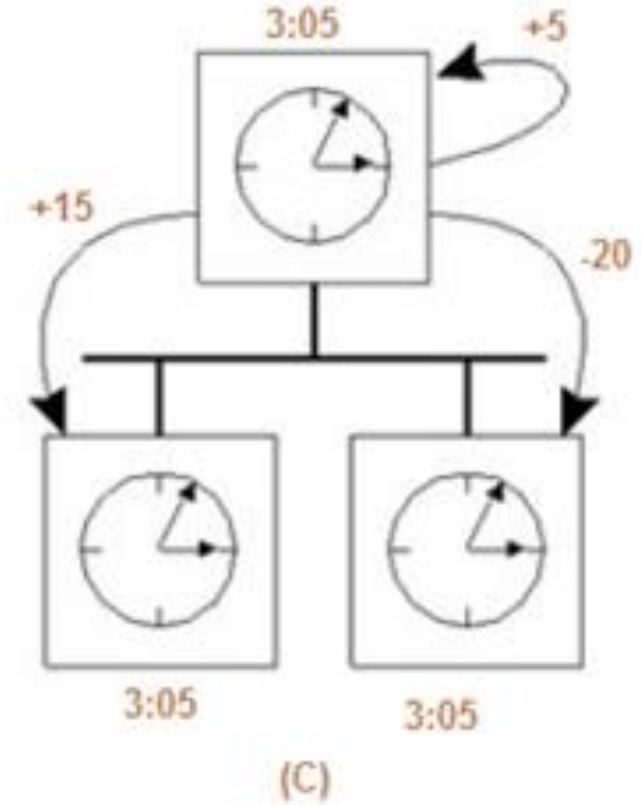
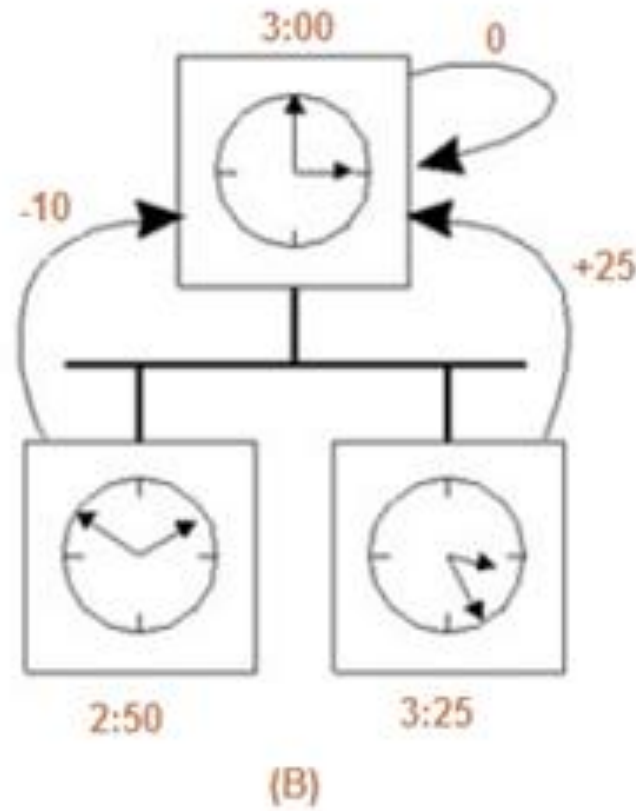
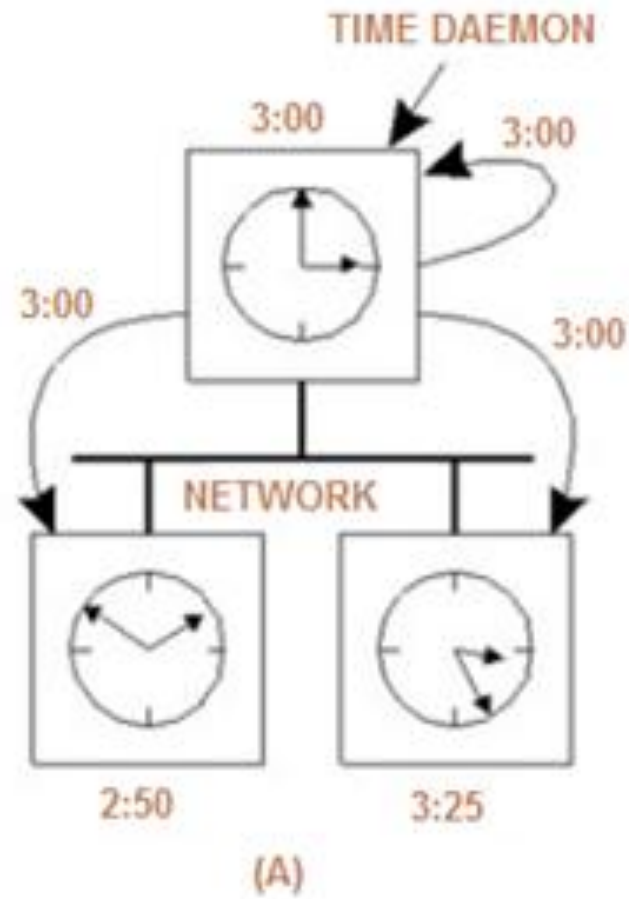
(b)

The Berkeley Algorithm (3)

- Time daemon tính toán ra giá trị trung bình và gửi cho các máy giá trị cần hiệu chỉnh



The Berkeley Algorithm (4)

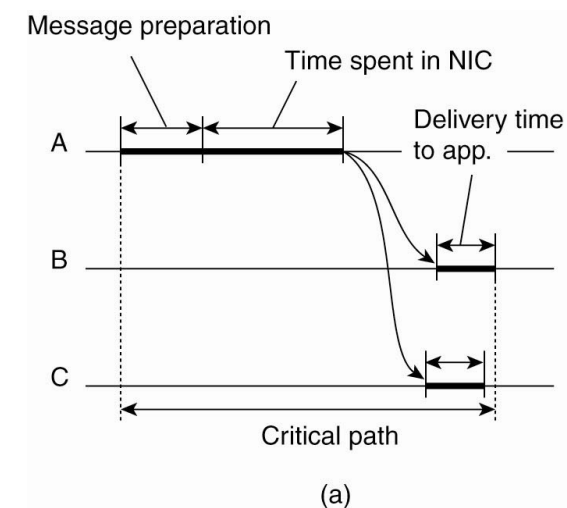
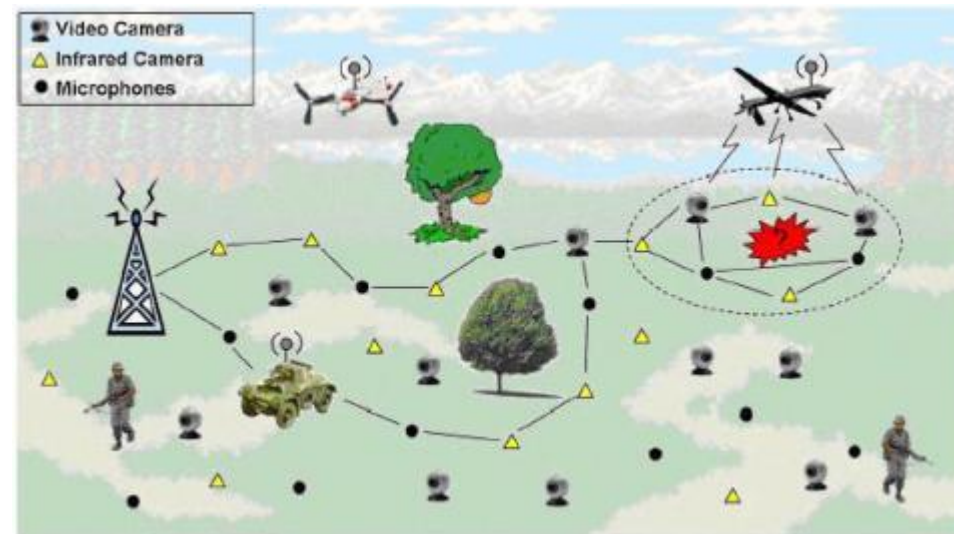


Đồng bộ hóa ĐHVL trong các mạng không dây (1)

Ví dụ trong mạng cảm biến không dây (WSN)

- Sử dụng pin
- A,B,C là các nút
 - A đóng vai trò là nút quảng bá

=> Tối ưu, tránh tốn kém năng lượng



Đồng bộ hóa ĐHVL trong các mạng không dây (2)

□ RBS (Reference Broadcast Synchronization)

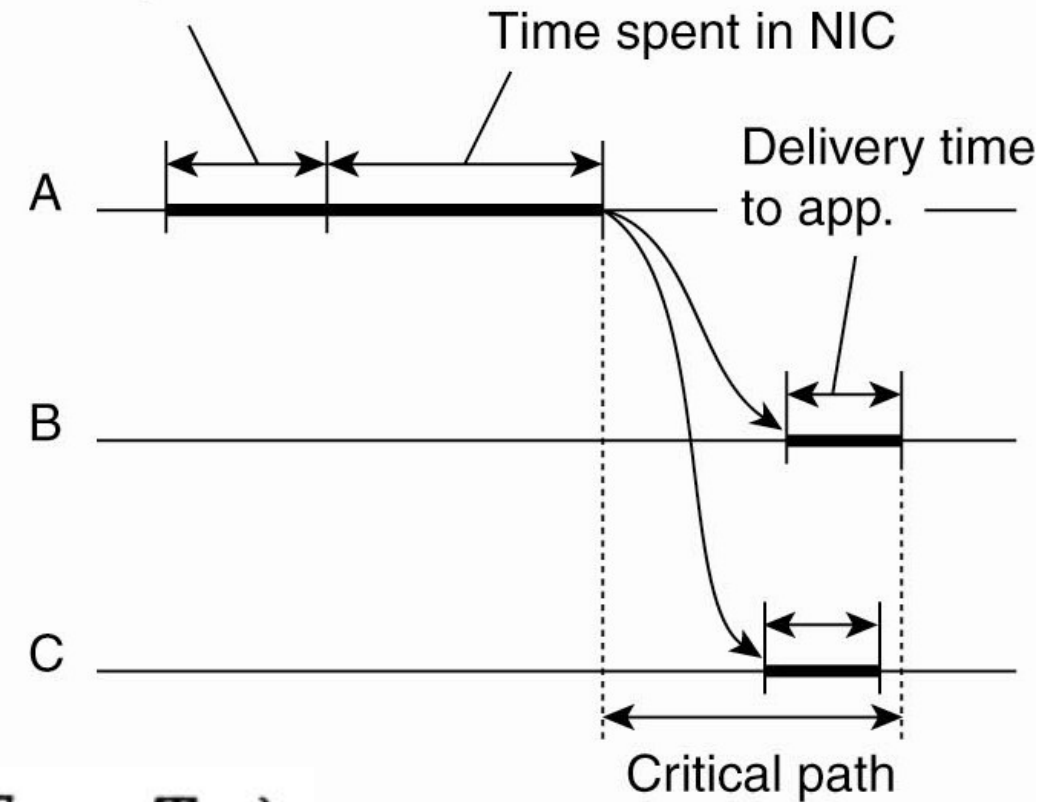
Trung bình cộng của M lần hiệu độ lệch của p,q so với k



Offset[p,q]

$$Offset[p,q] = \frac{\sum_{k=1}^M (T_{p,k} - T_{q,k})}{M}$$

Message preparation



(b)

2. Đồng bộ hóa đồng hồ logic

- Cơ chế đồng bộ hóa đồng hồ logic của Lamport
- Vector clocks

2.1. Cơ chế ĐBH đồng hồ logic của Lamport (1)

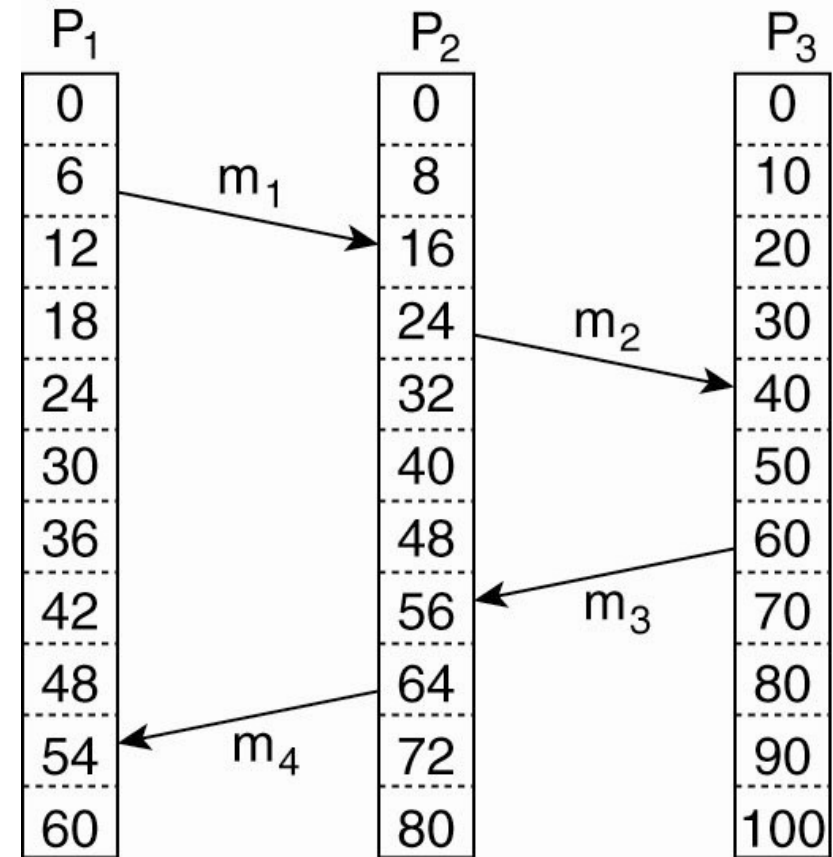
- Mỗi quan hệ “xảy ra trước” (happens-before) $\rightarrow$ diễn ra với 2 ngữ cảnh:
 1. Nếu a và b là các sự kiện của cùng 1 tiến trình, và a xảy ra trước b , thì $a \rightarrow b$ là đúng.
 2. Nếu a là sự kiện gửi một thông điệp từ một tiến trình, b là sự kiện nhận của thông điệp đó ở 1 tiến trình khác, thì $a \rightarrow b$ là đúng
- Mỗi quan hệ bắc cầu: $a \rightarrow b$ và $b \rightarrow c$, thì $a \rightarrow c$
- Với các sự kiện tương tranh a và b thì không có $a \rightarrow b$ và cũng không có $b \rightarrow a$

Cơ chế ĐBH đồng hồ logic của Lamport (2)

Ví dụ:

- 3 tiến trình P1,P2,P3 nằm trên 3 máy khác nhau
- Đồng hồ vật lý mỗi máy là khác nhau
- Có 4 thông điệp (m1-m4) trao đổi giữa 3 tiến trình
 - m1,m2 vẫn đảm bảo theo cơ chế Lamport
 - m3,m4 xảy ra vấn đề

=> Cần cập nhật lại đồng hồ để đảm bảo nguyên lý Lamport : $t_{recv} > t_{send}$



(a)

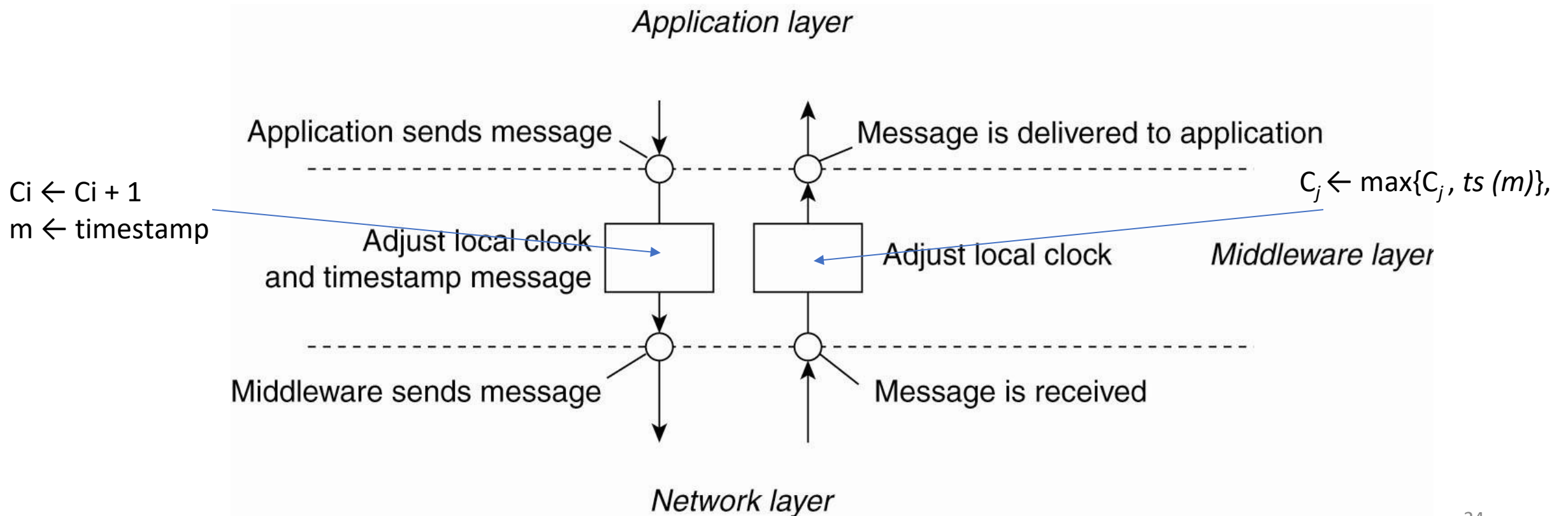
Cơ chế ĐBH đồng hồ logic của Lamport (3)

Chúng ta cần cập nhật bộ đếm C_i cho tiến trình P_i

1. Trước mỗi sự kiện, P_i thực thi: $C_i \leftarrow C_i + 1$.
2. Khi tiến trình P_i gửi thông điệp m tới P_j , nó sẽ đặt timestamp của m là $ts(m)$ bằng với giá trị C_i (sau khi thực hiện bước 1).
3. Khi nhận được thông điệp m , tiến trình P_j cập nhật lại giá trị bộ đếm cục bộ của mình:
 $C_j \leftarrow \max\{C_j, ts(m)\}$, sau đó sẽ chuyển thông điệp lên tầng ứng dụng.

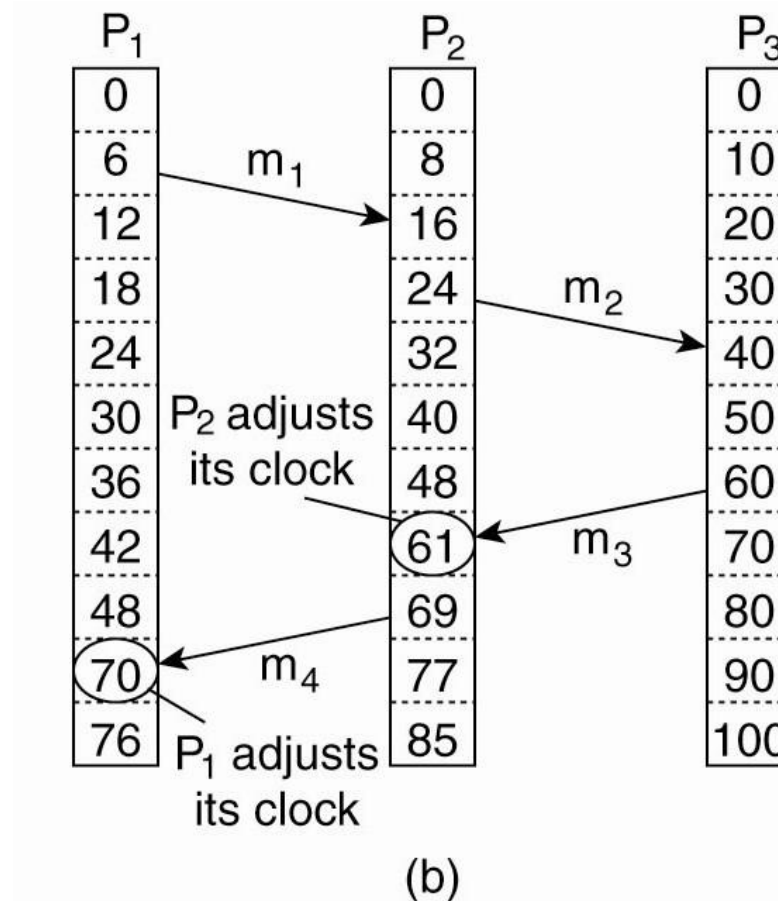
Cơ chế đồng hồ logic của Lamport (4)

- Figure 6-10. The positioning of Lamport's logical clocks in distributed systems.

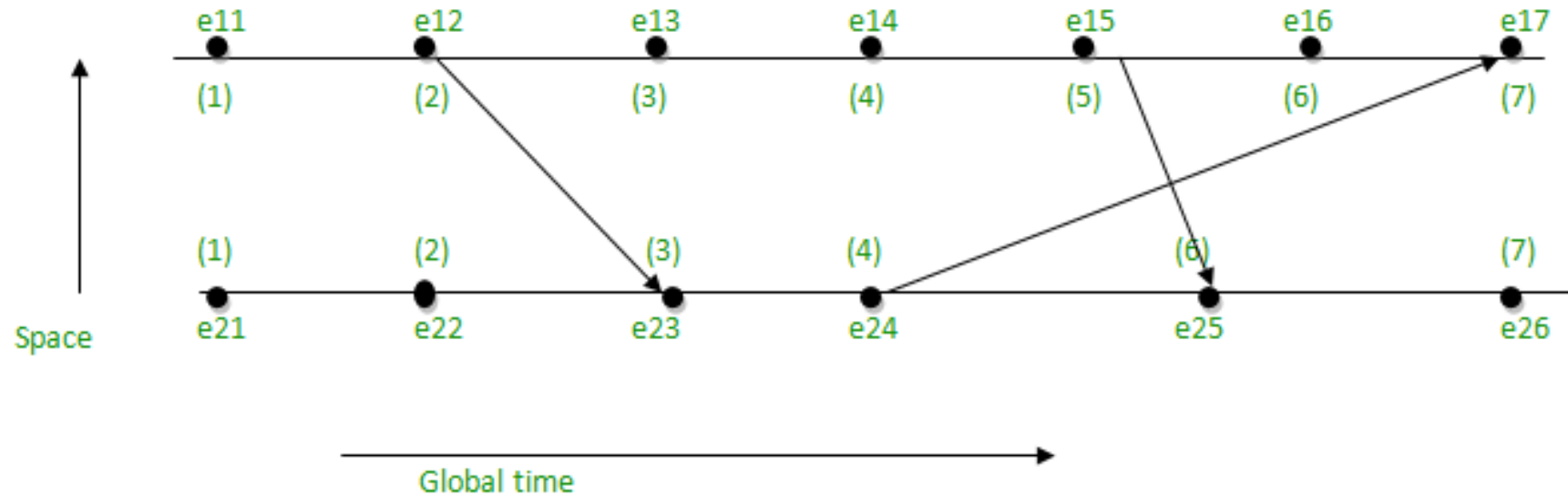


Cơ chế đbhh đồng hồ logic của Lamport (5)

(b) Giải thuật Lamport hiệu chỉnh lại các giá trị clock



Bài tập



$$e_{11} = 1$$

$$e_{21} = 1$$

The value of the next point will go on increasing by 1

Calculate value of e_{12} - e_{17} , e_{21} - e_{26}

Bài tập

Three computers, A, B, and C communicate using a protocol that implements Lamport logical clocks (they include their clock time stamp in messages). At the beginning of time, all three computers begin with their logical clock set to zero. Later, the following sequence of events occurs:

- A sends message M1 to B
- C sends message M2 to B
- B receives M2 before M1
- B replies message M3 to C first, and then message M4 to A
- After receiving message M3, C sends message M5 to A
- After receiving message M5, A sends message M6 to B
- After receiving message M6, B sends message M7 to A
- The last message A receive is M4

Indicate the time included with the messages as they are sent at each step.

Send(M1, _) Send(M2, _) Send(M3, _) Send(M4, _) Send(M5, _) Send(M6, _) Send(M7, _)

Bài tập

Có 3 máy tính ở PKA, bao gồm A,B, và C, giao tiếp với nhau sử dụng một giao thức mà được triển khai theo cơ chế đồng hồ logic của Lamport

Tại thời điểm đầu tiên, cả ba máy bắt đầu với đồng hồ logic được thiết lập = 0

Sau đó:

- A gửi thông điệp M1 cho B: “Xin chào”
- Sau khi nhận M1, B gửi thông điệp M2 cho C: “A nói với tôi rằng: Xin chào”
- Sau khi nhận M2, C gửi thông điệp M3 cho A: “B đang chán nản”

Điền vào chỗ trống thời gian được đánh kèm trong thông điệp ở mỗi lần gửi

Send(M1,); Send(M2,); Send(M3,)

Tiếp tục:

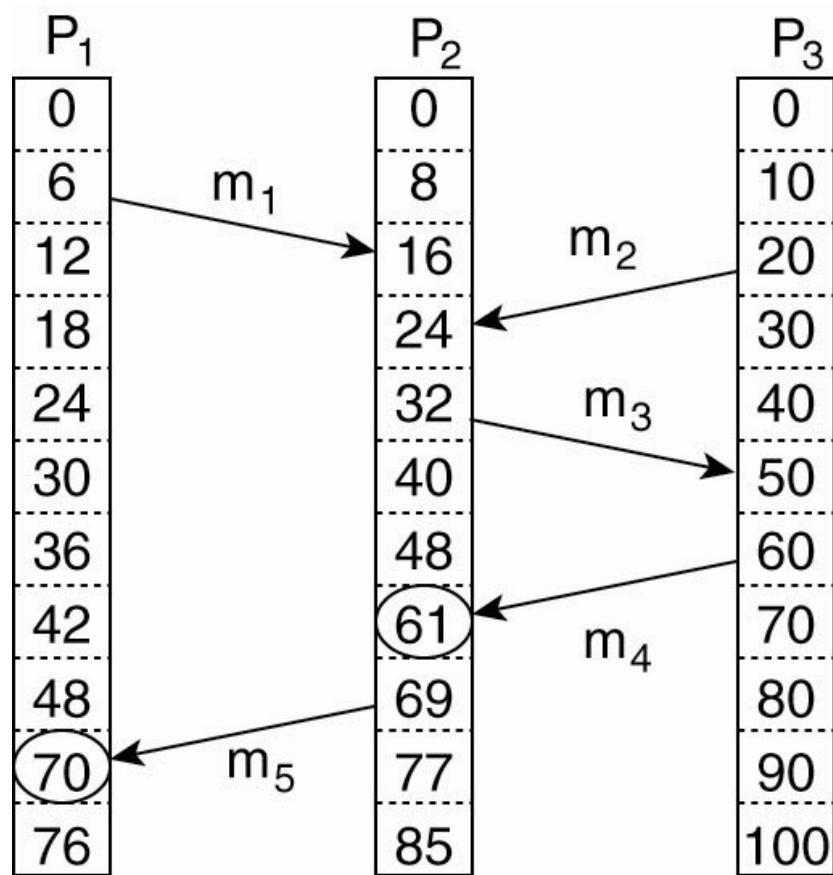
- Sau khi nhận M3, A gửi thông điệp M4 cho B: “C thật đơn điệu”
- Sau khi nhận M4, B gửi thông điệp M5 cho A: “C đang chán nản”
- A nhận M5

Sau toàn bộ quá trình trên, đồng hồ logic ở mỗi máy hiển thị:

A -; B -; C -

2.2. Vector Clocks (1)

- Việc truyền thông điệp có tính tương tranh sử dụng đồng hồ logic



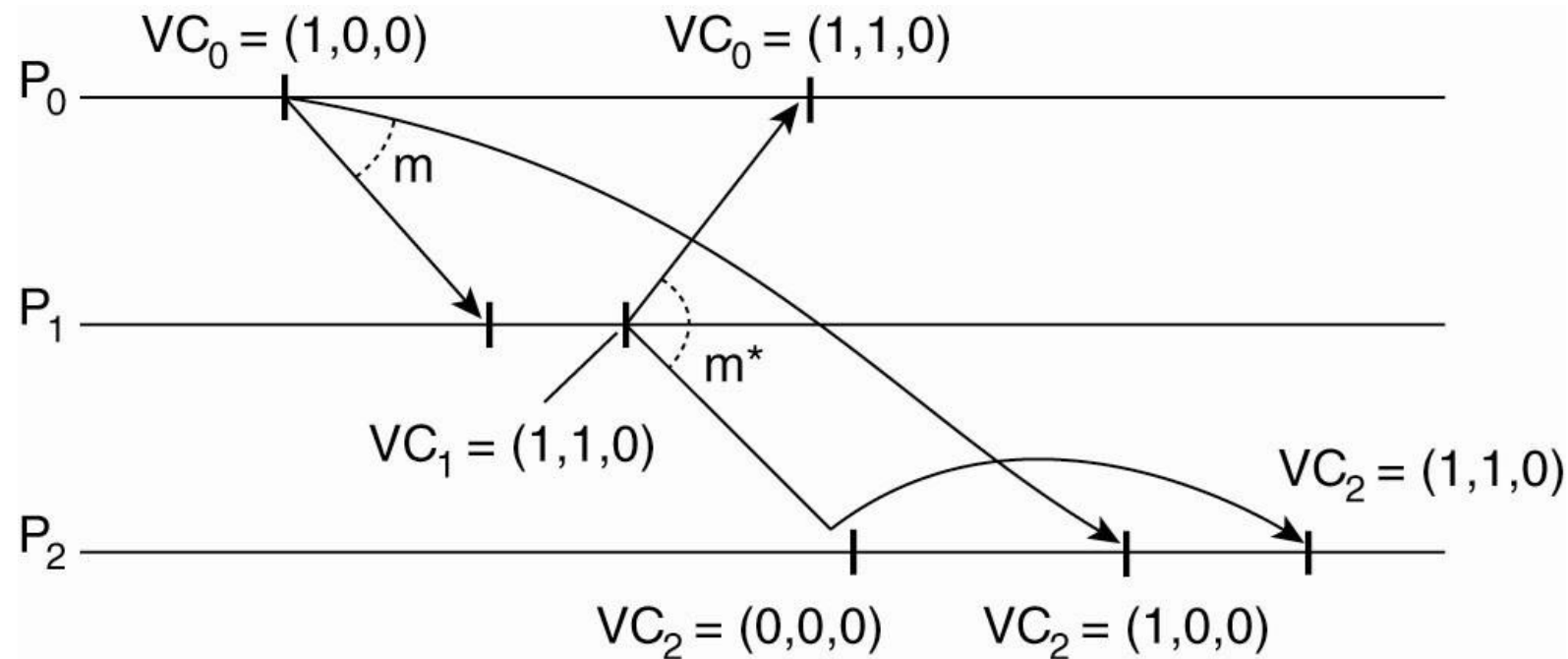
Vector Clocks (2)

- Véc-tơ clock được xây dựng bằng cách để mỗi tiến trình P_i quản lý một véc-tơ VC_i với 2 tính chất sau:
 1. $VC_i[i]$ là số các sự kiện đã xảy ra ở P_i . nói cách khác, $VC_i[i]$ là đồng hồ logic cục bộ của P_i .
 2. Nếu $VC_i[j] = k$ thì P_i biết rằng đã có k sự kiện xảy ra ở P_j . Là tri thức của P_i về thời gian cục bộ ở P_j .

Vector Clocks (3)

- Những bước cập nhật để đảm bảo tính chất 2:
 1. Trước mỗi sự kiện, P_i thực thi: $VC_i[i] \leftarrow VC_i[i] + 1$.
 2. Khi P_i gửi thông điệp m tới P_j , nó đặt véc-tơ timestamp $ts(m)$ vào m bằng với VC_i sau khi thực hiện bước 1.
 3. Khi nhận được 1 thông điệp m , tiến trình P_j cập nhật lại véc-tơ của mình:
 $VC_j[k] \leftarrow \max\{VC_j[k], ts(m)[k]\}$ với mọi k , sau đó mới gửi thông điệp lên tầng ứng dụng.

Thực thi trao đổi thông tin có tính nhân quả



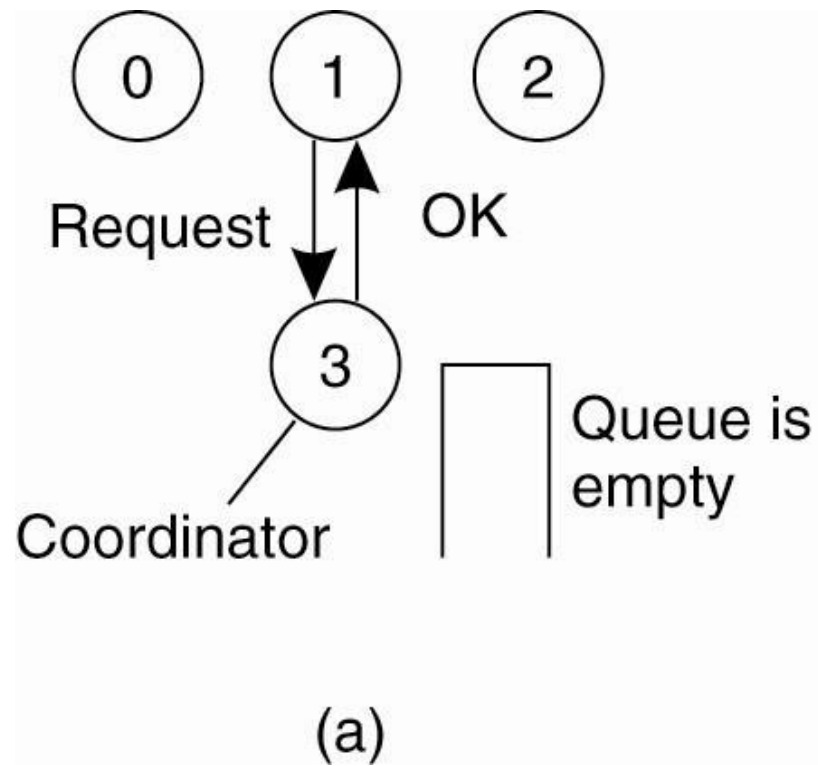
2 điều
kiện:

1. $ts(m)[i] = VC_j[i] + 1$
2. $ts(m)[k] \leq VC_j[k]$ for all $k \neq i$

3. Các thuật toán loại trừ lẫn nhau

- Giải thuật tập trung (Centralized Algorithm)
- Giải thuật phân tán (Distributed Algorithm)
- Giải thuật vòng (TokenRing Algorithm)

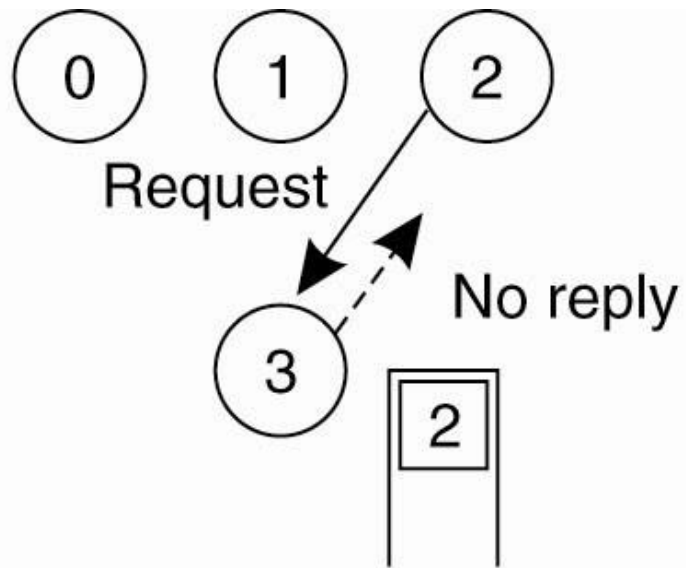
3.1. Giải thuật tập trung (1)



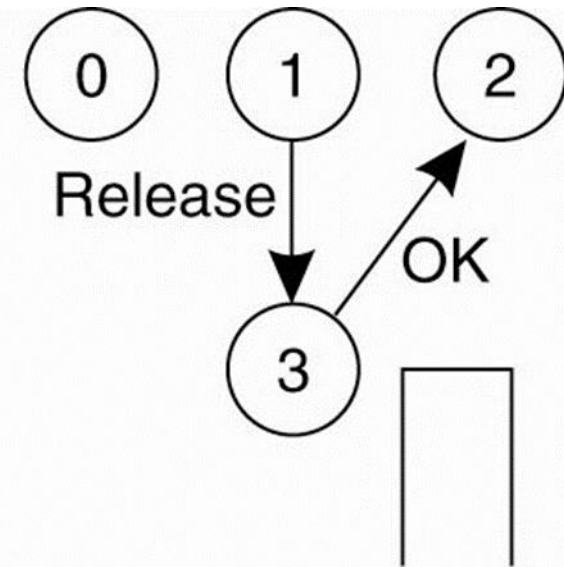
Giả sử mỗi tiến trình có một ID duy nhất.

Tiến trình được bầu chọn làm điều phối (Coordinator) là tiến trình có số ID cao nhất.

Giải thuật tập trung (2)



(b)



(c)

Giải thuật tập trung (3)

Ưu điểm:

- Giải thuật đảm bảo sự loại trừ lẫn nhau: bộ điều phối chỉ để một tiến trình truy cập tài nguyên trong 1 thời điểm.
- Không có tiến trình nào phải đợi vô thời hạn.
- Dễ thực thi vì chỉ yêu cầu 3 thông điệp (yêu cầu – cho phép – giải phóng) cho mỗi lần sử dụng tài nguyên

Nhược điểm:

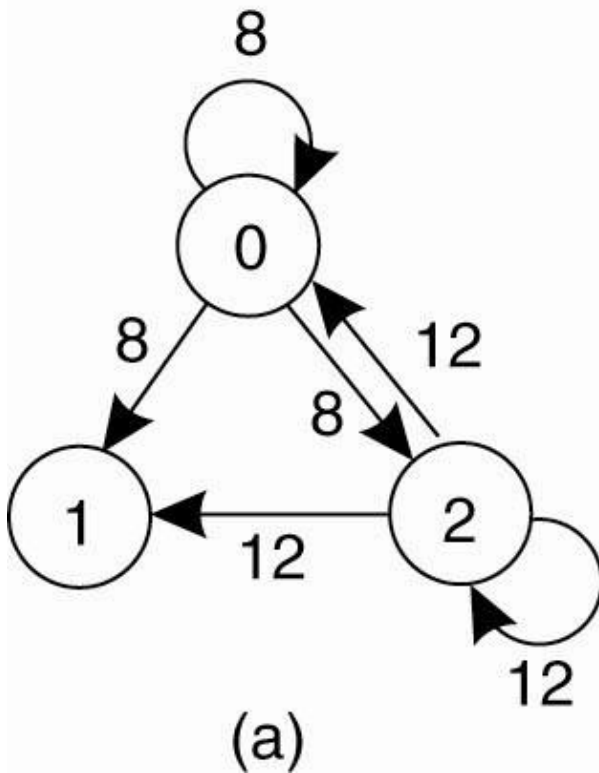
- Nếu tiến trình điều phối bị hỏng => hệ thống sụp đổ .
- Đối với hệ thống lớn, nếu chỉ có một tiến trình điều phối sẽ dẫn tới hiện tượng thất cổ chai

3.2. Giải thuật phân tán (1)

- Muốn vào dùng tài nguyên => quảng bá thông điệp request (bao gồm tên của tài nguyên, số xử lý của nó và thời gian theo logic hiện tại)
- Với mỗi tiến trình nhận được request:
 1. Nếu đang không dùng và không muốn dùng TN: Trả lời OK
 2. Nếu đang dùng: không trả lời và lưu vào hàng đợi
 3. Nếu đang không dùng nhưng cũng muốn dùng: so sánh timestamp
- Chỉ vào dùng TN khi nhận đủ các OK của các tiến trình khác

Giải thuật phân tán (2)

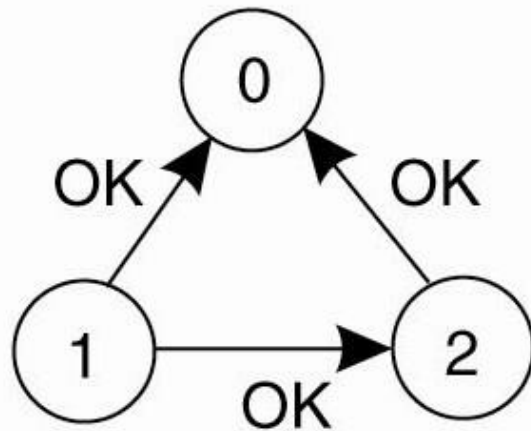
Ví dụ



- Tiến trình 0 gửi 1 yêu cầu với timestamp = 8 đến tất cả các tiến trình.
- Trong cùng thời điểm đó, tiến trình 2 cũng làm tương tự với timestamp = 12

Giải thuật phân tán (3)

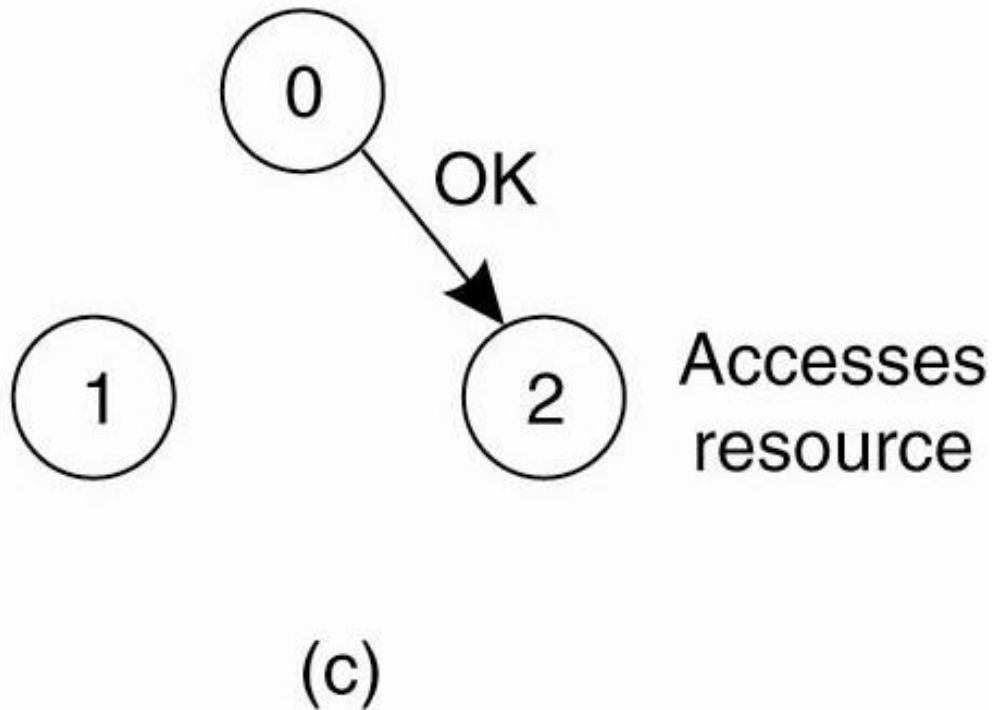
Accesses
resource



(b)

- Tiến trình 1 không muốn truy cập tài nguyên, vì thế nó gửi OK đến cả 2 bên gửi.
- Cả tiến trình 0 và 2 nhận ra sự xung đột và cùng so sánh timestamp.

Giải thuật phân tán (4)

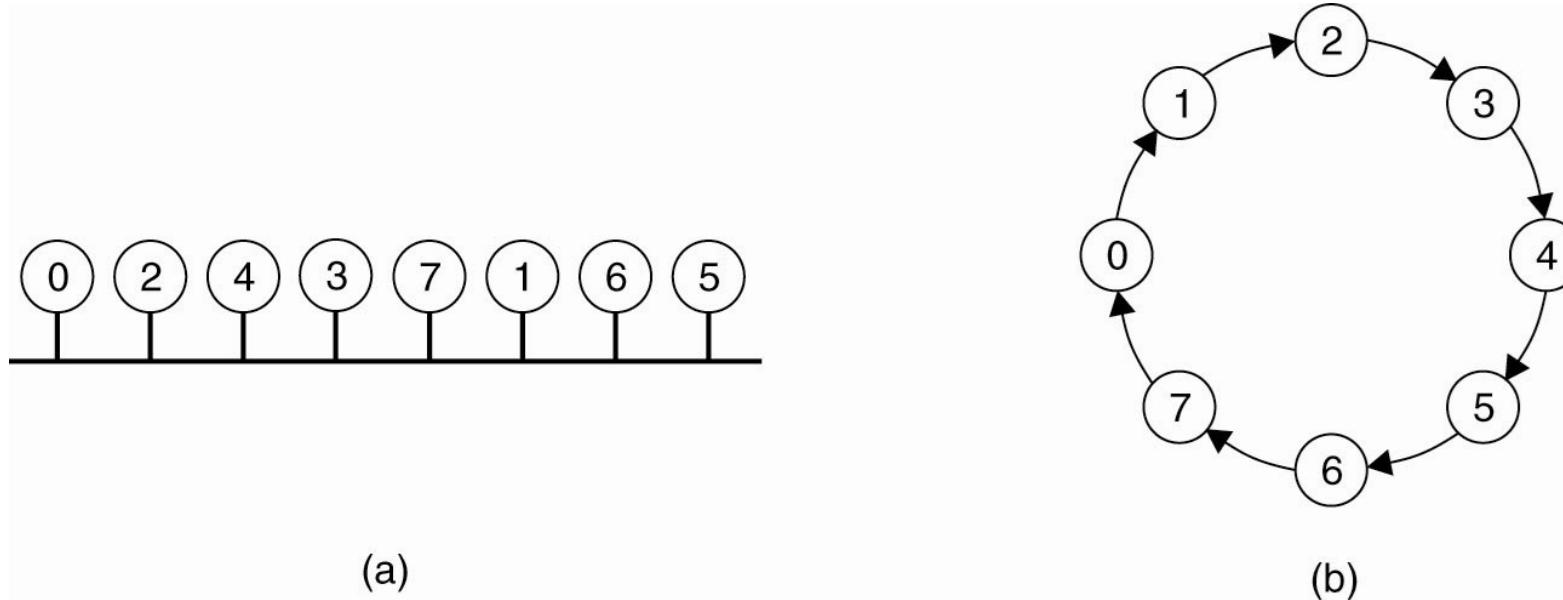


- Tiến trình 0 truy cập vào tài nguyên và xếp tiến trình 2 vào hàng đợi của 0 để xử lý và truy cập tài nguyên.
- Sau khi kết thúc, 0 loại bỏ yêu cầu từ tiến trình 2 khỏi queue của nó và gửi thông điệp OK đến tiến trình 2 và cho phép 2 thực hiện.

Giải thuật phân tán (5)

- Khi tổng số lượng tiến trình trong hệ thống là n thì yêu cầu $2(n-1)$ thông điệp cho mỗi thực thể .
- Nếu bất cứ 1 tiến trình nào lỗi, nó sẽ gây lỗi thông điệp phản hồi yêu cầu và khiến toàn bộ các tiến trình tiến vào vùng giới hạn
- Chậm, phức tạp và chi phí đắt so với thuật toán tập trung

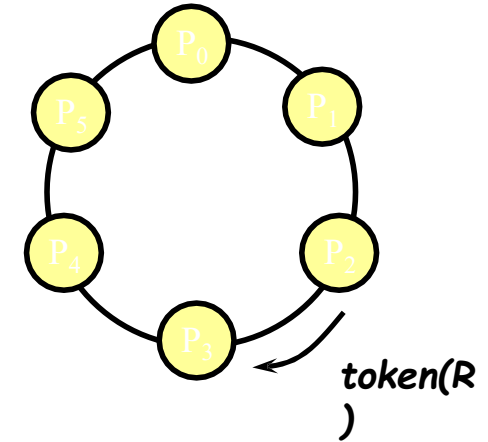
3.3. Giải thuật Token Ring



Giả sử tất cả các tiến trình được sắp xếp trên một vòng tròn logic, các tiến trình đều được đánh số và đều biết các tiến trình cạnh nó.

Token Ring algorithm

- Khởi đầu
 - Tiến trình P0 có token để vào sử dụng TN
- Token được truyền đi trong vòng topo
 - Từ P_i truyền đến $P_{(i+1) \bmod N}$
- Khi một tiến trình nhận được token:
 - Tự kiểm tra xem có muốn vào dùng TN không
 - Nếu không, truyền token cho nút kế tiếp
 - Nếu có, giữ lại token. Dùng xong thì truyền tiếp đi



4. Các giải thuật bầu chọn

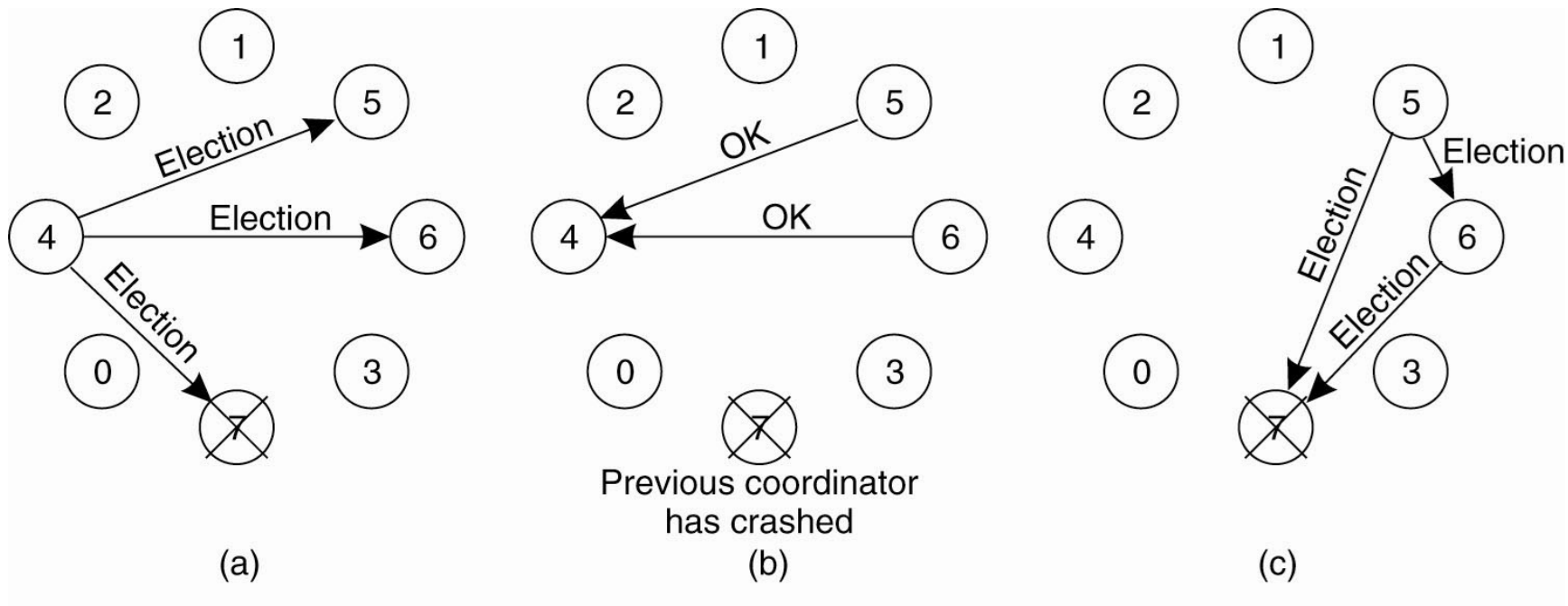
- Các giải thuật truyền thống
 - ▣ Giải thuật áp đảo (Bully)
 - ▣ Giải thuật vòng (Ring)
- Các giải thuật bầu chọn cho mạng không dây
- Các giải thuật bầu chọn cho mạng diện rộng

Các giải thuật bầu chọn

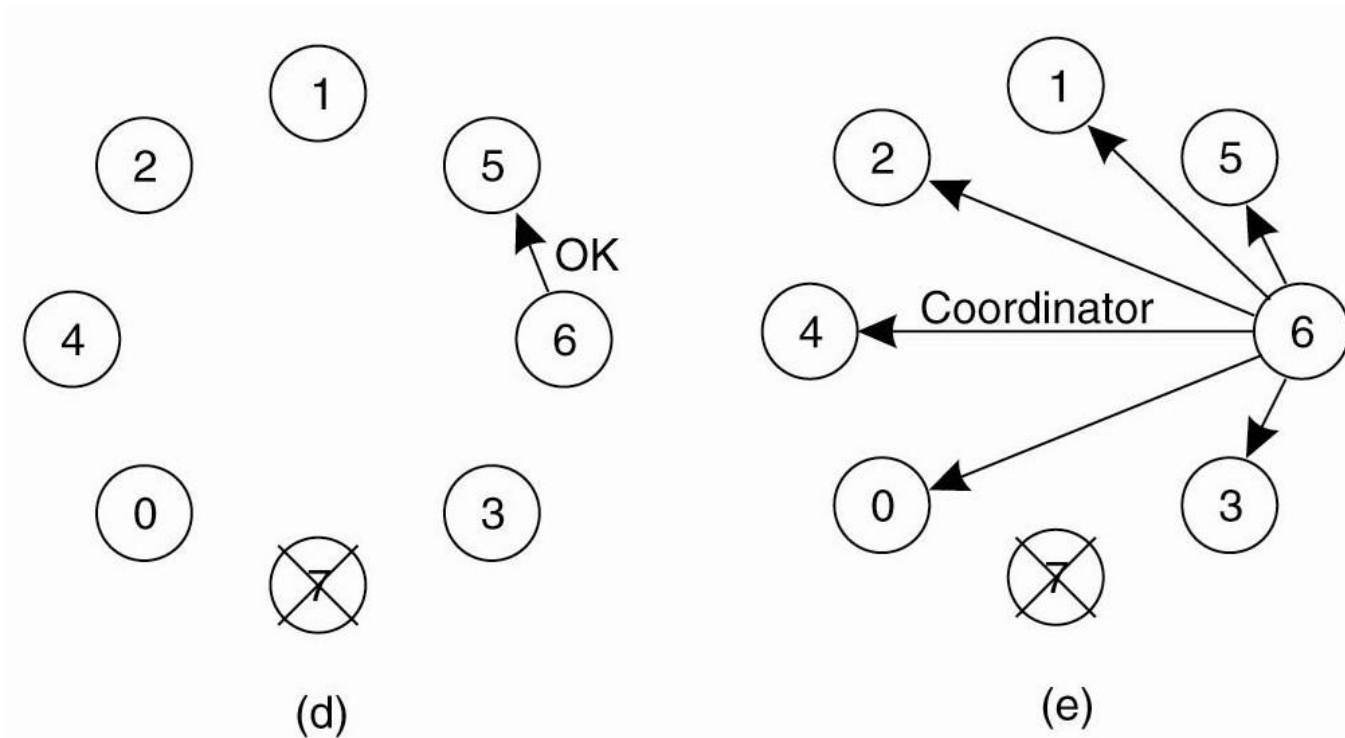
□ Giải thuật Bully

1. P gửi thông điệp *ELECTION* cho tất cả các tiến trình có ID lớn hơn mình
 1. Nếu không có tiến trình nào trả lời (sau timeout), P biết mình được chọn
 2. Chỉ cần 1 tiến trình trả lời, P biết mình không được chọn và dừng lại.
2. Quá trình trên lặp lại với các tiến trình nhận được *ELECTION*

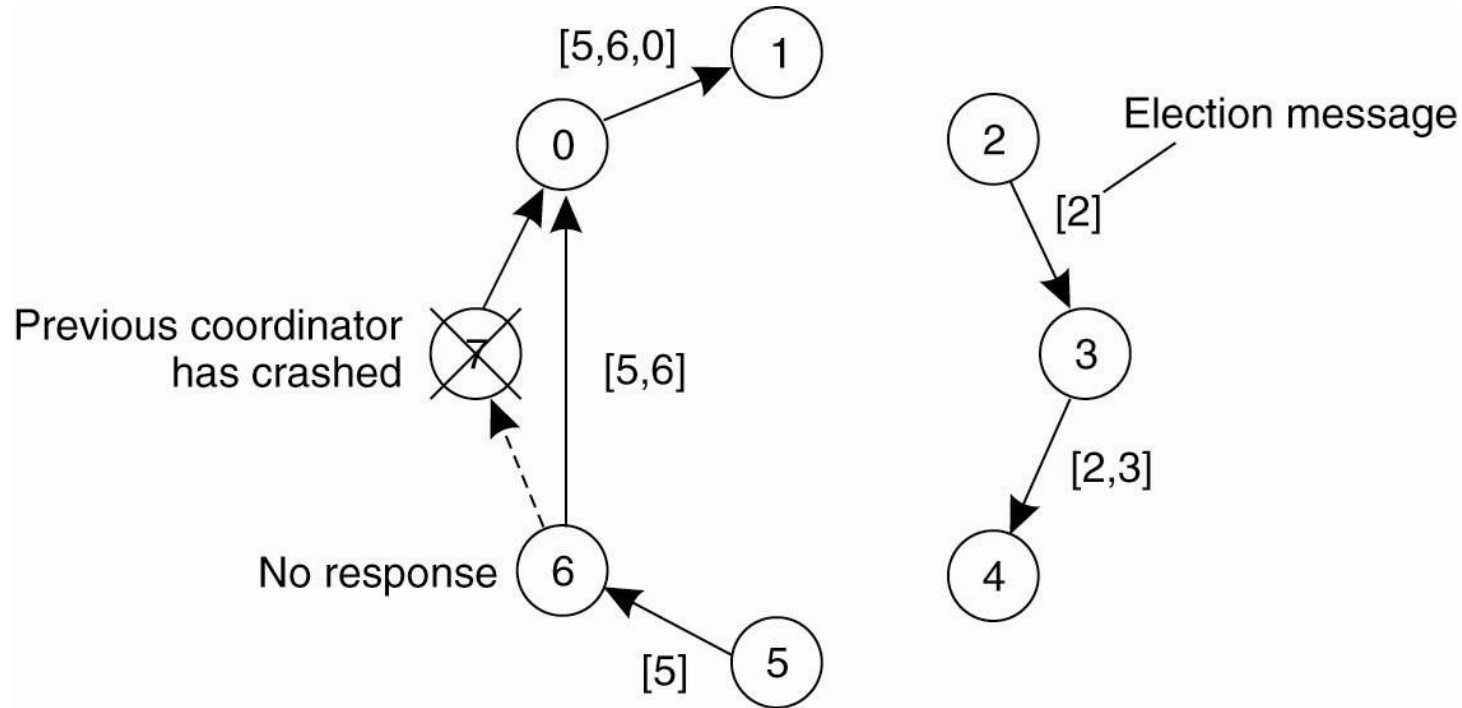
Giải thuật BULLY (1)



Giải thuật BULLY (2)

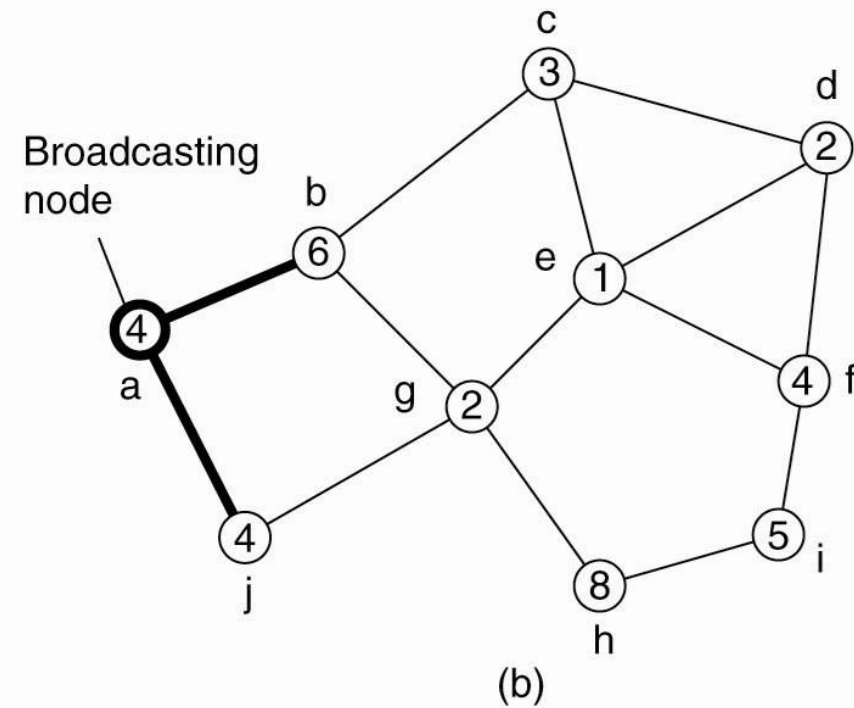
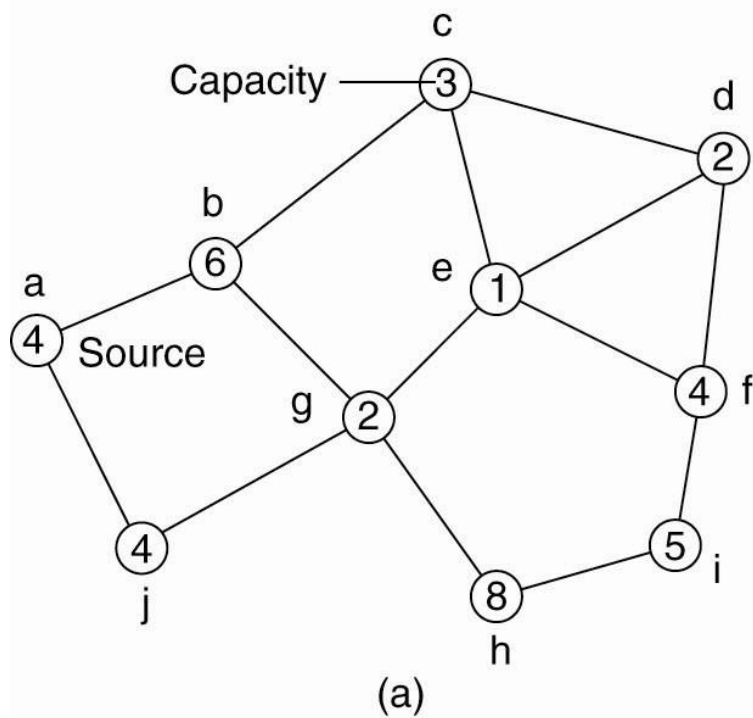


Giải thuật RING

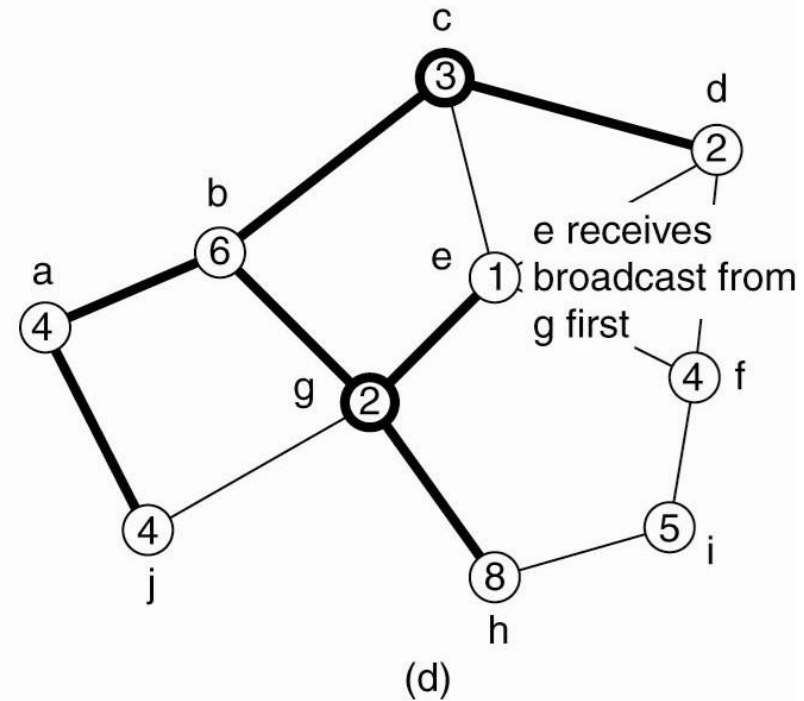
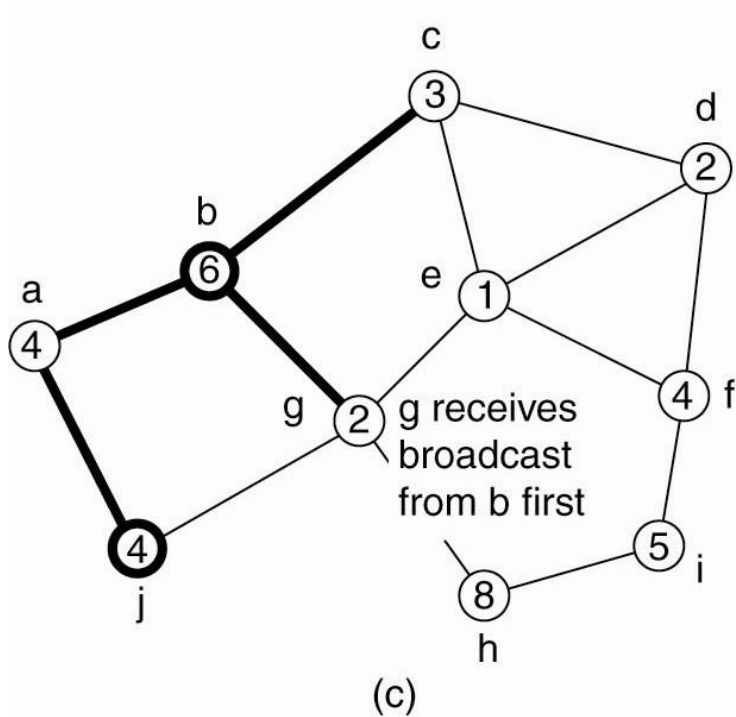


- Các tiến trình có một ID duy nhất và được sắp xếp trên 1 vòng tròn Logic.
- Mỗi một tiến trình có thể nhận biết được tiến trình bên cạnh mình.

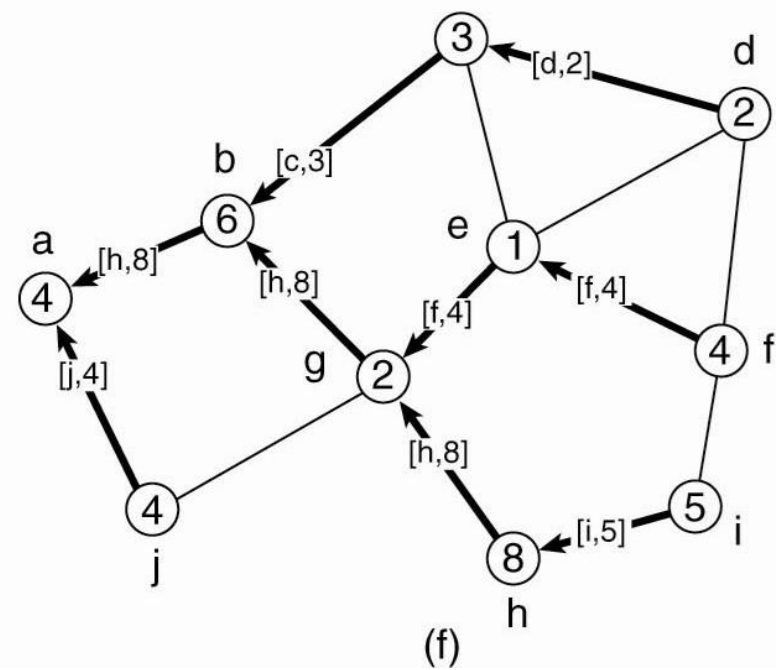
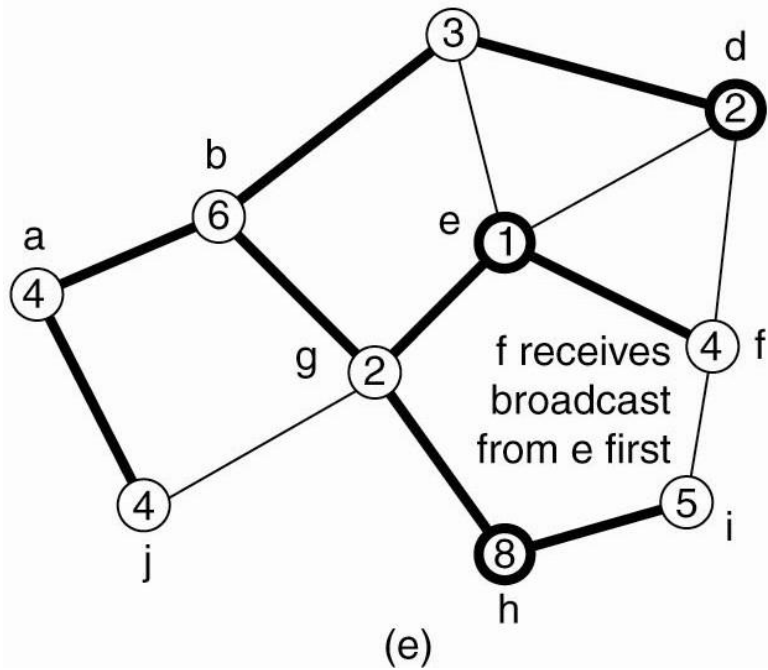
Giải thuật bầu chọn cho mạng không dây (1)



Giải thuật bầu chọn cho mạng không dây (2)



Giải thuật bầu chọn cho mạng không dây (3)



Bầu chọn cho mạng cỡ lớn (1)

- Yêu cầu cho các giải thuật chọn *superpeers*:
 1. Các nút thường khi truy cập đến các superpeers phải có độ trễ thấp
 2. Các superpeers phải được phân tán đều trong toàn bộ mạng overlay
 3. Phải định trước tỷ lệ các nút superpeers trong tổng số các nút trong mạng
 4. Mỗi nút superpeers không được phục vụ quá nhiều nút thường

Bầu chọn cho mạng cỡ lớn (2)

